

Homework 09: An Expression Evaluator

Model arithmetic expressions as a sealed hierarchy, then let the compiler catch a case you forget on purpose.

This is the classic use for sealed types plus records plus pattern matching, and it is worth writing once by hand.

Part One: The Model

Create `Calculator.java` in the classic form.

Define a `sealed interface Expr` permitting five record types:

Record	Components
<code>Literal</code>	an <code>int</code> value
<code>Add</code>	two <code>Expr</code>
<code>Multiply</code>	two <code>Expr</code>
<code>Negate</code>	one <code>Expr</code>
<code>Divide</code>	two <code>Expr</code>

Note that `Add`, `Multiply`, `Negate` and `Divide` hold `Expr`, not `int`. That is what makes the structure a tree and the methods recursive.

Part Two: Three Operations

Write three methods, each a **single switch expression with no default**:

1. `evaluate(Expr)` returning the `int` result
2. `print(Expr)` returning a readable string, parenthesising binary operations
3. `countNodes(Expr)` returning how many nodes the tree contains

Build `(3 + 4) * -(2)` and run all three against it.

Use **record patterns** to pull out the components. Write `case Add(Expr left, Expr right)`, not `case Add a` followed by `a.left()`.

Part Three: Prove The Compiler Is Watching

Add a sixth record, `Power`, to the `permits` clause. Do **not** update `evaluate`.

Compile it. Record the exact error, including the missing pattern it names, in a comment. Then either handle `Power` or remove it again.

Write one sentence on what would have happened instead if `evaluate` had a `default` branch.

Part Four: Simplify

Write `simplify(Expr)` returning a simpler but equivalent `Expr`, using **guards** for these rules:

Rule	Example
<code>0 + x</code> and <code>x + 0</code>	<code>(0 + 7)</code> becomes <code>7</code>
<code>1 * x</code> and <code>x * 1</code>	<code>(1 * 9)</code> becomes <code>9</code>
<code>0 * x</code> and <code>x * 0</code>	<code>(0 * 9)</code> becomes <code>0</code>
<code>--x</code>	<code>--5</code> becomes <code>5</code>
constant folding	<code>(2 + 3)</code> becomes <code>5</code>

The double-negation rule is the interesting one. It wants a **nested** record pattern, matching two levels in a single case label.

Ordering matters here. Say in a comment why the guarded cases must come before the unguarded case for the same type.

Acceptance Criteria

- ☐ Runs with `java Calculator.java`
- ☐ `Expr` is `sealed` with an explicit `permits` clause
- ☐ `evaluate`, `print` and `countNodes` are each one switch with **no default**
- ☐ Record patterns are used to destructure, not accessor calls
- ☐ A comment quotes the real "missing patterns" error you produced
- ☐ A sentence explains what `default` would have cost
- ☐ `simplify` implements all five rules using guards
- ☐ The double-negation rule uses a nested record pattern
- ☐ A comment explains the guarded-before-unguarded ordering

Stretch

Division by zero cannot be handled by returning an `int`. Model the outcome as a second sealed interface, `Result`, permitting `Success(int value)` and `Failure(String reason)`, and write `evaluateSafely` returning it.

Then write a comment comparing this to throwing an exception. Module 10 covers exceptions next, so note what you think each approach costs the caller before you read it.

Hint, if the nested pattern will not compile

`Negate` holds an `Expr`, and `Negate` is itself an `Expr`. So a `Negate` whose component is another `Negate` is matched by naming the type twice and destructuring both. Write the outer pattern first and let the inner one sit inside its parentheses.